Indian Institute of Science, Bengaluru
Department of Computational and Data Sciences
DS252: Cloud Computing | August–December 2026

Ask IISc

A Privacy-Preserving Retrieval-Augmented Assistant for Academic Policy and Admissions Information over Self-Hosted Open-Weight Language Models

Project Report: Architecture, Ingestion Pipeline and Cloud Inference Framework

Team:	ASKS	**Date:**	04 November 2026
Version:	1.0	**Reference GPU target:**	single 24 GB GPU

Team Members

Kshitij Pratihast	SR# 28768	kshitijprat1@iisc.ac.in
Aditya Barodia	SR# 28351	adityabarod1@iisc.ac.in
Smit Patel	SR# 26782	smitpatel@iisc.ac.in
Sejadri Banik	SR# 26387	sejadribanik@iisc.ac.in

Abstract

Students, applicants and faculty at the Indian Institute of Science (IISc) must locate academic rules, course schedules and admissions criteria across many separately maintained web pages and PDF notices. Sending these questions, together with the institutional documents needed to answer them, to a commercial LLM API conflicts with institutional data-governance requirements. This report describes **Ask IISc**, a retrieval-augmented question-answering service built on open-weight language models (Llama 3.2 3B, Llama 3.1 8B and a 4-bit AWQ variant of the 8B model) served with vLLM.

We describe (1) a versioned ingestion pipeline with SHA-256 content hashing, a PostgreSQL version registry and snapshot-gated activation; (2) hybrid retrieval that combines dense search in Qdrant (BGE-M3), a typed knowledge graph backed by Neo4j with a vector-only fallback, and cross-encoder reranking; (3) an admission controller with per-key token buckets, a per-tenant concurrency ceiling ($C = 20$), a bounded FIFO queue ($Q = 10$) and HTTP 429 load shedding; (4) a serving tier with a prefix-cache-friendly prompt layout, token streaming, similarity-gated abstention, citation verification and a source-excerpt fallback; and (5) a memory-budget analysis for a single 24 GB GPU together with infrastructure-as-code for a demonstration deployment on AWS. The evaluation on 120 held-out questions (answer accuracy and citation correctness against tokens/s, GPU-hours and p95 latency for the 3B, 8B and quantized variants) is in progress and is reserved in Section **??**. We state explicitly where the AWS demonstration deployment departs from the zero-egress design target.

Contents

1 Introduction and Problem Formulation **1**

1.1 The Institutional Information Gap 1

1.2 The Privacy Mandate: Zero Institutional Data Egress 1

1.3 The Cloud-Systems Problem: Resource-Constrained Inference 1

2 Architectural Contributions **2**

3 End-to-End System Architecture and Tier Placement **3**

3.1 Physical Tier Decomposition 3

3.2 Architectural Layout Diagram 4

3.3 Data Flows: Ingestion (Write Path) and Serving (Read Path) 4

4 Detailed Subsystem Design and Implementation **5**

4.1 Document Ingestion, Parsing and Staleness Detection 5

4.2 Versioned Snapshots and Activation 6

4.3 Hybrid Knowledge Representation: Qdrant and Neo4j 6

4.4 Multi-Tenant Admission Control and Load Shedding 7

4.5 Multi-Tenant GPU Inference with vLLM 9

4.6 Grounded Generation, Calibrated Abstention and Attribution 10

5 Hardware Budgeting and Memory Allocation Analysis **11**
5.1 Purpose and Assumptions . . . 11
5.2 Memory Budget Formulation . . . 11
5.3 Sequence-Length Design Point . . . 11
5.4 Results for the Three Configurations . . . 12
5.5 What Remains to Be Measured . . . 12

6 Infrastructure as Code, Deployment Topology and Security **12**
6.1 Deployment Profiles . . . 12
6.2 Role-Based Scope Enforcement . . . 13

7 Engineering Challenges **14**
7.1 Room Numbers Parsed as Course Codes . . . 14
7.2 Wrong Citation Headings and Stale Terms . . . 14
7.3 Unsafe Write Order During Re-Indexing . . . 14
7.4 Admission Controller Ignored Its Queue . . . 14
7.5 Graph Evidence Lost in Prompt Packing . . . 15

8 Evaluation and Experimental Results **15**
8.1 Workload Generator . . . 15
8.2 Baseline Configurations . . . 16
8.3 Experiment E1: Model Variant Comparison . . . 16
8.4 Experiment E2: Concurrency and Admission Control . . . 18
8.5 Experiment E3: Retrieval and Grounding . . . 19
8.6 Experiment E5: Graph-Assisted Retrieval . . . 19
8.7 Experiment E5: Graph-Assisted Retrieval Evaluation . . . 19
8.8 Baseline Evaluation . . . 21
8.9 Peak VRAM Usage . . . 22
8.10 Experiment E6: Degraded-Mode Behaviour . . . 22
8.11 Overall Findings . . . 22

9 Requirement Traceability Matrix **22**

10 Team Contribution Breakdown **23**

11 Conclusion **24**

List of Figures

1 Ask IISc end-to-end system topology and tier boundaries (placeholder). 4
2 Model Variant vs Grounded Answer Accuracy 16
3 Model Variant vs Throughput 17
4 Concurrent Users vs p50/p95/p99 Latency (8B Model) 18
5 Vector-only (B2) vs Graph-assisted (Full System) Retrieval on Connected Multi-hop Questions 20

List of Tables

1 Data crossing each tier boundary. 4
2 vLLM launch variants (`deploy/vllm_variants.sh`); all use `-max-model-len 8192`. 9
3 Estimated KV-cache pool and concurrency on a 24 GiB GPU ($L_s = 3{,}200$ tokens, $M_{\text{ovh}} = 2.0$ GiB). 12
4 Deployment profiles and status of NFR-8 (zero institutional data egress at run time). . . . 13
5 Roles, authentication and accessible document scopes. 13
6 Admission probe with synthetic work and no model calls (`bench/admission_probe.py`), before and after the fix. 15
7 Grounded Answer Accuracy 17
8 Model Throughput 17
9 Latency and TTFT Measurements 18
10 Concurrency Latency Results 18
11 Retrieval and Grounding Metrics 19
12 Vector-only vs Graph-assisted Retrieval 19
13 Connected Question Benchmark 20
14 Dedicated Graph-RAG Evaluation 20
15 Graph Link Ablation Study 21
16 Dedicated Graph Retrieval Evaluation 21
17 B0 Evaluation 21
18 B1 vs Full-System Throughput 21
19 Peak VRAM Usage vs 24 GB Budget 22
20 Degraded Mode Results 22
21 Functional requirements, implementing artifacts and verification. 22
22 Subsystem ownership and contributions per team member. 24

1 Introduction and Problem Formulation

1.1 The Institutional Information Gap

The Indian Institute of Science (IISc), established in 1909, comprises over forty academic departments, interdisciplinary centers, and administrative divisions. Academic adminis tration at the Institute is governed by a decentralized corpus of digital documentation: the General Information Handbook, Scheme of Instructions, Student Academic Guidelines, Division-specific Degree Requirements(e.g., EECS, MechanicalSciences), and seasonal Admissions Bulletins. Every semester three user groups meet administrative friction:

1. **Prospective applicants :** Confronted with multi-tiered eligibility criteria across dozens of specialized graduate programs (M.Tech, M.Tech Research, Ph.D., Integrated Ph.D.), where qualifying GATE disciplines, undergraduate degree prerequisites, and written test/interview modalities vary significantly between departments.
2. **Enrolled students :** Subject to stringent academic milestones, such as course add/drop deadlines, minimum cumulative grade point averages (CGPA), credit rollover limits, comprehensive examination timeframes, and leave ordinances.
3. **Faculty and administrative staff :** Over burdened by repetitive email inquiries regarding routine procedural policies,leading to operational communication bottlenecks and administrative latency during peak periods.

Existing search relies on keyword matching over web pages and static PDFs. Answering a compound question, for example *"Which DS courses are listed as prerequisite alternatives for DS 256?"* or *"Do DS 252 and DS 256 have the same instructor?"*, requires reading several pages and connecting facts between them.

1.2 The Privacy Mandate: Zero Institutional Data Egress

The obvious remedy is a retrieval-augmented generation (RAG) assistant [1] backed by a commercial proprietary API. This is unsuitable in an institutional setting because the inputs include (i) documents that are not public, such as student-only and faculty-only circulars and schedules, and (ii) user queries and access patterns, which can reveal a student's academic situation or research intent. We therefore adopt a design invariant, referred to in the project requirements as **NFR-8** (*Zero Institutional Data Egress*): document parsing, indexing, graph modelling, prompt assembly and token decoding execute on infrastructure controlled by the Institute, using open-weight models whose weights are downloaded once at set-up time.

Section 6 reports honestly which deployment profiles satisfy this invariant today. In particular, the AWS demonstration deployment forwards retrieved evidence to an external model provider and therefore does *not* satisfy NFR-8; the self-hosted GPU configuration is the reference design that does.

1.3 The Cloud-Systems Problem: Resource-Constrained Inference

Removing the commercial API forces the use of self-hosted 3–8 billion parameter models in an interactive, multi-tenant setting, which raises four systems problems:

1. **Accelerator memory ceiling.** The target is a commodity single-GPU node with 24 GB of memory . Llama 3.1 8B in BF16 needs about $8.03 \times 10^9 \times 2 \approx 16$ GB (14.96 GiB) for weights alone, leaving

a narrow margin for the CUDA context, activations and the Key–Value (KV) cache (Section 5).

2. **Memory-bandwidth-bound decoding.** Prefill is compute-bound, but every decoded token streams all model weights through the memory bus. Serving concurrent users without iteration-level batching either serialises requests or wastes the accelerator.
3. **KV-cache fragmentation.** Allocating contiguous KV memory for the maximum sequence length wastes a large share of GPU memory; Kwon et al. report that existing systems use only 20–38% of KV memory for actual token states [2], which caps the achievable batch size.
4. **Heterogeneous state.** Academic information is partly prose and partly relational (prerequisite chains, instructor–course–term links). Keeping a crawler, a vector database, a property graph and a relational registry mutually consistent without serving stale or mixed-version facts needs an explicit versioning protocol.

These constraints turn what might look like an applied NLP task into a cloud-systems problem of resource allocation, admission control and distributed state.

2 Architectural Contributions

Ask IISc is an end-to-end service rather than a wrapper around an existing framework. Its principal contributions are:

1. **Hybrid retrieval with a typed knowledge graph.** Dense similarity search (Qdrant with BGE-M3 embeddings) is combined with a typed knowledge graph (courses, instructors, venues, prerequisites, research areas, labs, programme and policy facts) stored in Neo4j. Graph expansion is bounded, every edge carries the chunk, version and access scope of its evidence, and a cross-encoder reranks the merged candidates. Graph evidence reaches the prompt as structured provenance rather than as extra text appended after vector hits.
2. **Version-aware, snapshot-gated retrieval.** Every indexed chunk carries a `version_id`. A registry snapshot lists the active version identifiers and the embedding model; retrieval filters vectors and graph edges against that list inside the database query, and re-audits every returned passage. A benchmark can pin a snapshot (`pinned_snapshot`, HTTP 409 if it is no longer active).
3. **Admission control sized to the serving tier.** A per-key token bucket, a per-tenant concurrency ceiling C, a bounded per-tenant FIFO queue Q with a maximum wait, and a global concurrency cap shed excess load with HTTP 429 and a `Retry-After` header. Slots are held for the full streamed response.
4. **Prefix-cache-friendly prompt layout.** The system message and the opening of the user message are constant, so consecutive requests share an identical token prefix that vLLM's Automatic Prefix Caching can reuse. All request-specific content follows the prefix.
5. **Perimeter access control.** Role-based scopes (`public`, `student`, `faculty`, `admin`) are applied as mandatory filters in the vector search and in graph traversal, so out-of-scope passages never reach the prompt.
6. **Grounded generation with calibrated abstention and graceful degradation.** Evidence sufficiency is gated by a calibrated reranker score before the model is called; answers must cite numbered sources which are verified after generation; a failed graph degrades to vector-only retrieval; a failed model endpoint degrades to source excerpts.

3 End-to-End System Architecture and Tier Placement

Asynchronous batch ingestion is decoupled from low-latency question answering. Components are placed in three tiers separated by explicit network boundaries and contracts.

3.1 Physical Tier Decomposition

3.1.1 Device / Client Tier

The client is a single static web page (`askiisc_serving/static/index.html`) using plain HTML, CSS and JavaScript with no build step. It

- sends the user's question, optional programme and academic-year filters, and a bearer token or session cookie over HTTPS;
- consumes Server-Sent Events (SSE) incrementally and renders the bracketed source markers (`[S1]`, `[S2]`, ...) as links to source excerpts;
- keeps conversation history only in page memory; each follow-up is rewritten as a standalone question and answered from freshly retrieved sources, so earlier answers are never treated as evidence.

The device never receives embeddings, graph records or out-of-scope content. It receives only in-scope excerpts and, for signed-in users, permitted attachments.

3.1.2 Edge / Gateway Tier

The gateway is a FastAPI application (`askiisc_serving/app.py`, `auth.py`, `email_auth.py`, `admission.py`) that

- authenticates requests, using either API keys (programmatic access and benchmarks) or e-mail one-time-password sessions restricted to `@iisc.ac.in`; unauthenticated browsers receive the `public` scope only;
- applies admission control (Section 4.4) and returns HTTP 429 with `Retry-After` when capacity is exceeded;
- exposes `POST /v1/ask`, `GET /v1/sources/{chunk_id}`, `/healthz`, `/readyz` and a Prometheus `/metrics` endpoint.

3.1.3 Private Core Tier

The core tier hosts the following components.

1. **Ingestion and retrieval services (CPU).**
   - *Ingestion workers*: crawl, parse, chunk, embed and index (Section 4.1).
   - *Version registry*: PostgreSQL tables `document_versions`, `document_chunks` and `active_snapshots`.
   - *Raw snapshot store*: MinIO object storage for retrieved source files (the registry records the object key).
   - *Knowledge stores*: Qdrant (dense vectors) and Neo4j Community Edition (typed graph).
   - *Embedding and reranking*: BGE-M3 via a Text Embeddings Inference (TEI) server or in-process, and a BGE cross-encoder reranker.

- *Retrieval/serving orchestrator*: the FastAPI process that combines retrieval, prompt assembly and citation checking; retrieval can run in-process or as a separate HTTP service (`POST /retrieve`).

2. **GPU node.** A node with the NVIDIA accelerator hosts the vLLM OpenAI-compatible server (one model variant at a time, Section 4.5) and communicates only with the orchestrator over the internal network.

3.2 Architectural Layout Diagram

Figure 1 will show the complete topology, tier boundaries and the data crossing each boundary (Table 1).

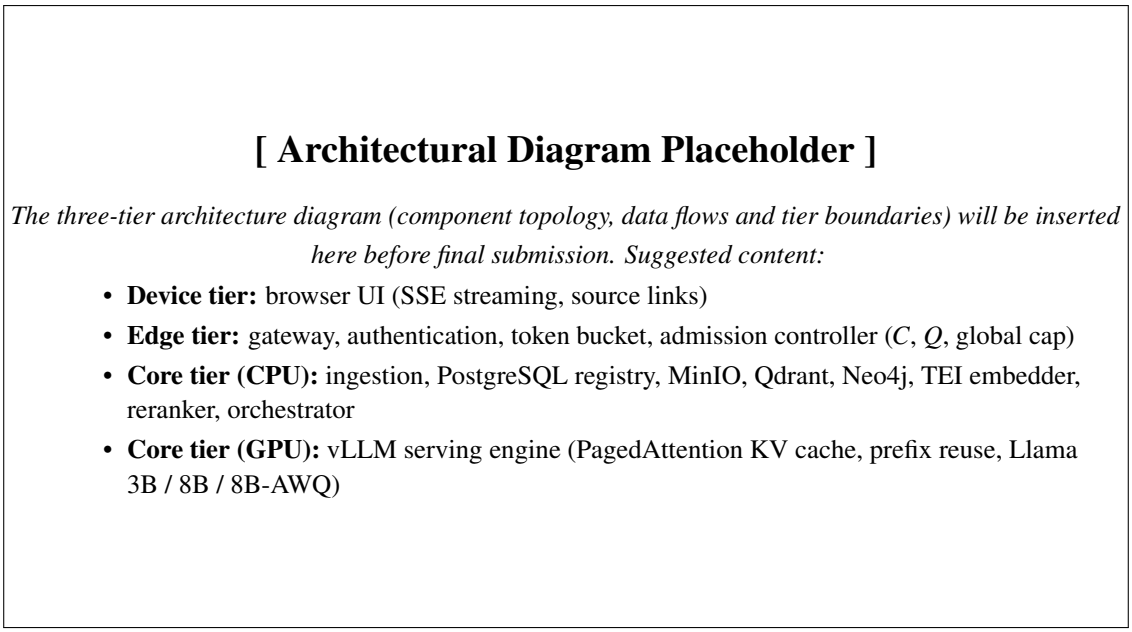


Figure 1. Ask IISc end-to-end system topology and tier boundaries (placeholder).

Table 1. Data crossing each tier boundary.

Boundary	Crosses inward / forward	Crosses back
Device ↔ Edge	Question text, optional programme/year filters, bearer token or session cookie (HTTPS)	Answer tokens (SSE), source markers and excerpts, status (`answered`, `abstained`, `fallback`)
Edge ↔ Core	Authenticated principal (tenant, role, scopes) and the question	Retrieval result (authorized passages, citations, trace)
Core (orchestrator) ↔ GPU node	Assembled prompt: system message, authorized evidence blocks, question	Generated tokens and token usage

3.3 Data Flows: Ingestion (Write Path) and Serving (Read Path)

Ingestion (write path). Ingestion runs on demand or periodically and never on the request path. It fetches the allow-listed seed URLs, parses HTML (and bounded PDF input) into sections, splits sections into chunks, computes content hashes, embeds the chunks, and writes vectors to Qdrant, edges to Neo4j and version and snapshot rows to PostgreSQL. A new version becomes visible to queries only when the active snapshot lists its `version_id`.

Serving (read path). A request passes authentication and admission control. Retrieval then (i) embeds the query, (ii) searches Qdrant with the version, scope and embedding-model filters, (iii) expands through the graph, (iv) reranks, and (v) decides whether the evidence is sufficient. If it is, the generator assembles the prompt, vLLM streams the answer, and the citation checker validates the result before the final status is returned.

4 Detailed Subsystem Design and Implementation

4.1 Document Ingestion, Parsing and Staleness Detection

The document ingestion pipeline converts unstructured web pages and static PDF files into clean, semantically dense, searchable text representations. The reference corpus is defined by `sources.yaml`. A separate importer (`tools/import_admissions.py`) adds a bounded snapshot of public postgraduate admissions pages from an explicit allow-list (`config/admissions_sources.json`).

4.1.1 Bounded Crawling

The crawler (`askiisc_retrieval/crawl.py`, `fetch.py`) is breadth-first with a maximum depth of two clicks from the seeds and a hard cap of 200 pages. It fetches HTML only, honours `robots.txt`, pauses 0.5 s between downloads, tracks visited URLs to avoid cycles, and saves each page once so that later runs are offline and reproducible. The corpus is defined by the site's own links, never by a question set. The admissions importer is stricter still: at most 14 source URLs, 8 MiB of source data, six PDF pages and 180 new chunks. It raises an error at a bound instead of truncating, writes to a new output directory, refuses to overwrite an existing one, and marks an incomplete build so that it cannot be activated.

4.1.2 Boilerplate Removal and Structure-Preserving Parsing

The parser (`parse.py`) removes non-content elements before extraction: scripts and styles, `nav`, `header`, `footer`, `aside`, navigation and content-info ARIA roles, breadcrumbs, sidebars and theme footer or menu blocks, followed by a last-line filter for residual footer phrases. Headings are retained as section labels so that each chunk has exactly one citable location. Schedule tables are converted to one chunk per row, with the course code taken from the Course ID column so that room numbers are not mistaken for courses (Section 7.1).

4.1.3 Chunking

Chunks (`chunking.py`) are sentence-aligned windows of 250 words with a 40-word overlap that never cross a heading; table rows stay one chunk each. The embedder truncates input at 512 tokens, which the chunk size stays well below.

4.1.4 Content Hashing and Change Tracking

The registry stores a SHA-256 content hash with every document version (`document_versions.content_hash`). For the normalised main-content text T of a fetched page,

$$H_{\text{current}} = \text{SHA-256}\big(\text{Normalize}(T)\big). \tag{4.1}$$

If H_{current} equals the stored hash, the page is treated as unchanged and is not re-embedded. If it differs, a new version row is created, the new chunks and edges are tagged with the new `version_id`, and the previous version is marked superseded. The raw source is archived in object storage and its key is recorded in the registry. Retrieved passages carry a freshness label, and evidence from a past academic term is labelled with that term in the prompt so that the model can say which term it refers to.

4.2 Versioned Snapshots and Activation

Combining vectors and a property graph risks *semantic skew*: a query could retrieve an updated course description together with a prerequisite rule from an outdated graph. We avoid this by making the active snapshot the single source of truth for what is queryable.

- The registry table `active_snapshots` records, for each completed ingestion run, a snapshot identifier, the graph version and the list of active version identifiers (`active_version_ids`). In the local stand-in this manifest is a JSON file read through the same `SnapshotProvider` interface.
- The retriever resolves the active snapshot and applies it *inside* the Qdrant search as a filter on `version_id` together with the access scope and the pinned embedding model. Graph edges carry the same `version_id`, scope and source chunk and are dropped if they do not match.
- Every returned passage is audited again against snapshot and scope before it can be cited; violations are dropped and counted.

Because old and new versions coexist in the stores and are separated only by the active-version filter, publishing a new snapshot switches what queries can see without a separate staging collection. The ordering of writes during re-ingestion matters for failure safety; Section 7.3 describes the defect we found and how it was fixed. A benchmark may send `pinned_snapshot` with each request and receives HTTP 409 if the corpus has changed, which keeps experiments reproducible.

4.3 Hybrid Knowledge Representation: Qdrant and Neo4j

4.3.1 Dense Retrieval

Dense retrieval (`stores.py`, `retriever.py`) uses the BAAI/bge-m3 model, which maps a chunk T to a unit-norm vector in $\mathbb{R}^{1024}$:

$$\mathbf{e} = \text{Embed}_{\text{BGE-M3}}(T) \in \mathbb{R}^{1024}, \qquad \|\mathbf{e}\|_2 = 1, \tag{4.2}$$

so cosine similarity between query vector $\mathbf{q}$ and passage vector $\mathbf{p}$ reduces to a dot product, $\text{Sim}(\mathbf{q},\mathbf{p}) = \mathbf{q}\cdot\mathbf{p}$. The snapshot records the embedding model, and the search filter requires it to match; a mismatch raises an error instead of mixing vector spaces (**FR-5**). Changing the model therefore requires re-embedding the corpus before a new snapshot is activated. The low-memory AWS demonstration uses the smaller BGE-small-en-v1.5 model (384 dimensions) on CPU with the reranker disabled (Section 6).

4.3.2 Retrieval Pipeline

For each question and set of scopes the retriever:

1. filters by active snapshot, user scope and embedding model inside the Qdrant search;
2. runs a course-code pass (a second search restricted to chunks tagged with a course named in the question) and a people pass for person or lab questions;

3. applies term logic: an explicit term in the question restricts results to that term; otherwise the latest term not after the current one is used for each course;
4. expands through the knowledge graph (below) and adds up to three supporting chunks;
5. reranks the top candidates and graph additions with a cross-encoder (**FR-13**), preferring chunks whose heading names the course asked about;
6. computes the evidence-sufficiency signal (Section 4.6);
7. audits every passage against snapshot and scope and attaches a numbered citation to each.

4.3.3 Relational Modelling in the Knowledge Graph

Vector search handles semantic matching, but multi-hop questions such as *"Which course does the instructor of DS 252 also teach?"* need explicit relations. The graph (`graph_neo4j.py`, `graph_rules.py`, `policy_graph.py`) uses the following schema:

- nodes: `Course`, `Faculty`/person, `ResearchArea`, `Lab`, `Department`; and, for policy questions, programme, offering and evidence-fact nodes;
- edges: `TAUGHT_BY`, `OFFERED_IN`, `HELD_IN`, `SCHEDULED_AT`, `EXAM_ON`, `REQUIRES` (prerequisite), `WORKS_ON`/`RESEARCHES_IN` and `HEADS`, plus typed programme and policy links with applicability qualifiers.

Every evidence edge must carry `chunk_id`, `version_id` and `source_url`; edges without a chunk are skipped and counted, and the *chunk*, never the edge, is what is cited.

The traversal rules, which are tested in one place, expand at most two hops and only through course and person nodes, with the same snapshot and scope filters; term rules ensure that only the requested or latest term is cited and that bridge edges are never cited. The policy graph connects a department's offering to its programme and to institute-level facts (fees, funding, duration, selection), with traversal bounded to three bridge hops, 512 states, 100 fact candidates and six supporting chunks; a programme such as M.Tech. (Research) has its own node so that its facts cannot fill gaps for the course M.Tech.

4.3.4 Evidence Selection and Fallback

For recognised relationship questions, `evidence_selection.py` chooses the anchor course and the graph endpoints that answer the requested relation, packs the required sources first and stops once the relationship bundle is complete. If a required bundle cannot fit within the prompt budget, the service abstains rather than answering from a partial bundle. Graph evidence reaches the prompt with its provenance (relation, endpoints and path), but the text sent to the model is always the source text of the cited chunks.

Resilience is explicit. If the graph is unavailable at start-up, or a graph call fails or exceeds its 0.5 s budget, retrieval returns vector-only results with `degraded=true`, which is surfaced in the response trace; if Neo4j becomes unreachable after a successful load, the last good in-memory copy continues to serve. A reranker failure keeps the vector order and never fails the request.

4.4 Multi-Tenant Admission Control and Load Shedding

To keep traffic bursts from driving the model server into queueing collapse, `askiisc_serving/admission.py` implements a multi-level gate in the gateway process.

4.4.1 Policy

Five parameters apply, with defaults from `settings.py`:

Symbol	Meaning	Variable	Default
R	per-API-key rate (requests/min)	`RATE_PER_MIN`	30
C	concurrent requests per tenant	`TENANT_CONCURRENCY`	20
Q	queued requests per tenant	`TENANT_QUEUE`	10
W	maximum wait in the queue (s)	`QUEUE_WAIT_S`	20
G	global concurrent requests per process	`GLOBAL_CONCURRENCY`	32

The rate limiter is a token bucket of capacity R refilled at $R/60$ tokens per second:

$$b(t) = \min\Big(R,\ b(t_0) + (t - t_0)\tfrac{R}{60}\Big), \tag{4.3}$$

and a request is admitted to the next stage only if $b \geq 1$, in which case $b \leftarrow b - 1$. Otherwise it is rejected with reason `rate_limited`. For a tenant with a active and w waiting requests and a process with g active requests overall, the decision for a request that has passed the rate limiter is

$$\text{decision} = \begin{cases} \text{admit}, & a < C,\ g < G,\ w = 0, \\ \text{enqueue (FIFO, at most } W \text{ s)}, & (a \geq C \text{ or } g \geq G \text{ or } w > 0) \text{ and } w < Q, \\ \text{reject: 429 } \texttt{tenant_queue_full}, & (a \geq C \text{ or } g \geq G \text{ or } w > 0) \text{ and } w \geq Q, \end{cases} \tag{4.4}$$

and an enqueued request that has not been admitted after W seconds is rejected with `queue_timeout`. With the queue disabled ($Q = 0$), global saturation returns `server_busy`. A single condition variable protects both limits, including the re-check after a waiter wakes, and a slot is held until the complete response or stream has finished. Every rejection is an HTTP 429 response with a `Retry-After` header and a JSON body carrying the `reason`, for example

```
HTTP/1.1 429 Too Many Requests
Retry-After: 2
{"error": "too many requests", "reason": "tenant_queue_full", "retry_after_s": 2}
```

4.4.2 Queueing Delay

If a tenant's C slots are all busy with mean service time S, slots free at rate C/S, so the j-th queued request waits about jS/C seconds, and the last of Q queued requests waits about

$$t_{\text{wait}}^{\max} \approx \min\Big(\tfrac{Q}{C}S,\ W\Big). \tag{4.5}$$

With $C = 20$ and $Q = 10$ this is $S/2$. Section 7.4 reports a live 31-request burst consistent with this behaviour: 20 active, 10 waiting, one request shed.

4.4.3 Why a Fixed Ceiling Is Still Needed

C is a latency and fairness control, not a memory-safety guarantee. vLLM preallocates its KV pool at start-up and schedules sequences against it, so KV growth does not cause run-time out-of-memory errors; when the pool is full, vLLM keeps requests waiting or preempts them. Admission control bounds how many requests can pile up behind the model server and returns an explicit, fast rejection instead of an unbounded wait. Section 5 quantifies how this interacts with each model configuration.

4.5 Multi-Tenant GPU Inference with vLLM

Generation uses vLLM's OpenAI-compatible server (`askiisc_serving/generator.py` is a thin streaming client; any OpenAI-compatible endpoint, including Ollama, MLX or a hosted API, can be substituted by changing two variables). Three optimisations matter for the cloud-systems questions.

4.5.1 PagedAttention

Contiguous per-request KV allocation wastes memory because actual lengths vary widely. PagedAttention [2] applies virtual-memory paging to the attention cache: the KV pool is divided into fixed-size blocks (16 tokens by default), a per-sequence block table maps logical positions to physical blocks, and blocks are allocated on demand as tokens are decoded, so waste is limited to the last partially filled block of each sequence.

4.5.2 Continuous (Iteration-Level) Batching

Static batching makes short answers wait for long ones. vLLM schedules at every forward pass: finished sequences leave the batch immediately and release their blocks, and newly arrived requests join for prefill. Baseline B1 disables batching with `--max-num-seqs 1`.

4.5.3 Automatic Prefix Caching (APC)

All requests share the same system instructions. The generator builds the prompt so that the system message is a constant and the user message begins with a fixed preamble; every request-specific part follows it:

$$P = \underbrace{[\text{system message}] \,\|\, [\text{“Evidence:”}]}_{\text{constant prefix}} \;\|\; \underbrace{[\leq 6 \text{ evidence blocks}]}_{\text{request-specific}} \;\|\; \underbrace{[\text{question and instruction}]}_{\text{request-specific}}. \tag{4.6}$$

With `--enable-prefix-caching`, vLLM hashes full token blocks and reuses the physical KV blocks of the constant prefix across requests, so that part of the prompt need not be prefilled again. The constant system message is about 1,500 characters (roughly 350–470 tokens depending on the tokenizer; the conservative estimator in the code gives 472), so only the whole 16-token blocks within the prefix are shareable. The gain in time-to-first-token and the cache hit rate are not asserted here; they are measured in the evaluation (Section **??**), with prefix caching disabled in baseline B1.

4.5.4 Model Variants and Launch Configuration

`deploy/vllm_variants.sh` launches one variant at a time on the 24 GB GPU (**FR-8**). `--gpu-memory-utilization` leaves room on the same GPU for the reranker.

Table 2. vLLM launch variants (`deploy/vllm_variants.sh`); all use `-max-model-len 8192`.

Variant	Checkpoint and precision	GPU util. u	Notes
`3b`	Llama-3.2-3B-Instruct, BF16	0.80	prefix caching on
`8b`	Llama-3.1-8B-Instruct, BF16	0.88	prefix caching on
`8b-awq`	Llama-3.1-8B-Instruct AWQ INT4 (`awq_marlin`)	0.80	prefix caching on
`b1-3b`	Llama-3.2-3B-Instruct, BF16	0.80	baseline B1: `--max-num-seqs 1`, prefix caching off

4.5.5 Prompt Budget and Streaming

The prompt budget is computed once, in the generator:

$$L_{\text{prompt}} = L_{\text{model}} - L_{\text{out}} - L_{\text{safety}} = 8192 - 384 - 128 = 7680 \text{ tokens}, \tag{4.7}$$

with at most six evidence blocks. Blocks are added best-first until the budget is spent, lower-ranked blocks are dropped, and a single oversized best block is cut. (The local Apple-silicon profile uses a 4,096-token context with 512 output tokens.) Streaming responses use SSE with three event types: `meta` (citation metadata, budget, snapshot), `token` and `done`. The `done` event carries the final status; if the post-hoc citation check fails, it sets `replace_streamed_text` and the client replaces the streamed text with source excerpts.

4.6 Grounded Generation, Calibrated Abstention and Attribution

4.6.1 Evidence-Sufficiency Gate

Language models answer plausibly even for unindexed topics. Before the model is called, retrieval computes the best cross-encoder relevance score over the reranked candidates,

$$s_{\max} = \max_i s_i, \tag{4.8}$$

and the evidence is judged sufficient only if $s_{\max} \geq \theta$:

$$\text{action} = \begin{cases} \text{generate a grounded answer,} & s_{\max} \geq \theta, \\ \text{abstain without calling the model,} & s_{\max} < \theta. \end{cases} \tag{4.9}$$

For the `bge-reranker-base` model the threshold is $\theta = 0.69$, calibrated on the development set after the crawl corpus was built: the highest score among must-abstain questions was 0.584 and the lowest among answerable ones was 0.797, so 0.69 lies in the gap. The threshold is never tuned on the held-out questions and must be recalibrated when the corpus or the reranker changes (for example for `bge-reranker-v2-m3` on the GPU). In the CPU-only AWS demonstration the reranker is disabled, so abstention there rests on the model's own instruction to abstain. On the 120-question set, retrieval alone would answer 3 of the 20 must-refuse questions because topic pages exist without the exact fact; abstention is therefore also enforced in the prompt (rule 3), and we measure abstention end to end rather than on retrieval alone.

The standard abstention reply is: *"I could not find this in the available sources."*

4.6.2 Citations and Verification

The system prompt requires every factual claim to carry a source marker such as `[S1]` or `[S1][S2]`, instructs the model to treat evidence text as data rather than instructions, to say which term evidence comes from when it is not current, and to abstain unless the evidence states the requested fact. After generation, `check_answer` verifies the markers: an answer that cites nothing, or cites a marker that was not supplied, fails the check. The JSON path regenerates once on a failed check; if the check still fails, or the model endpoint is unavailable, the response has status `fallback` and contains source titles, links and excerpts instead of generated text. `GET /v1/sources/{chunk_id}` resolves a marker to the full cited text, title, URL and version, but only if that chunk is active and within the caller's scope.

5 Hardware Budgeting and Memory Allocation Analysis

5.1 Purpose and Assumptions

This section sizes the three model configurations against a 24 GiB GPU and asks which concurrency each can sustain. All values are analytical estimates computed in binary units (1 GiB $= 2^{30}$ bytes), and the activation and CUDA overhead is an assumed constant. They are to be replaced or confirmed by the figures vLLM prints at start-up (`GPU KV cache size`, `Maximum concurrency`) and by its KV-cache-usage and preemption metrics during experiments E1 and E2. The values $C = 20$, $Q = 10$ and 30 requests/min are *starting values specified in the project requirements*, to be calibrated on the target GPU; they are not derived from the hardware. What the analysis provides is a feasibility check and an explanation of why the three configurations behave differently.

5.2 Memory Budget Formulation

The memory vLLM may use is a fraction u of GPU memory (`--gpu-memory-utilization`, Table 2); the remainder is left for the reranker and other processes. Within that share,

$$M_{\mathrm{KV}} = u\,M_{\mathrm{GPU}} - M_{\mathrm{w}} - M_{\mathrm{ovh}}, \qquad M_{\mathrm{w}} = N_{\mathrm{params}} \cdot b_{\mathrm{w}}, \tag{5.1}$$

where $M_{\mathrm{GPU}} = 24\,\mathrm{GiB}$, b_{w} is bytes per weight, and $M_{\mathrm{ovh}} \approx 2.0\,\mathrm{GiB}$ is an assumed allowance for the CUDA context and peak activations. The KV cache per token is

$$\kappa = 2 \cdot n_{\mathrm{layers}} \cdot n_{\mathrm{kv}} \cdot d_{\mathrm{head}} \cdot b_{\mathrm{kv}}, \tag{5.2}$$

where the leading 2 counts keys and values. Llama 3.1 8B has $n_{\mathrm{layers}} = 32$, $n_{\mathrm{kv}} = 8$ (grouped-query attention), $d_{\mathrm{head}} = 128$; Llama 3.2 3B has 28 layers with the same n_{kv} and d_{head}. The KV cache stays in 16-bit precision ($b_{\mathrm{kv}} = 2$) in all three configurations, since AWQ quantizes only weights:

$$\kappa_{8\mathrm{B}} = 2 \cdot 32 \cdot 8 \cdot 128 \cdot 2 = 131{,}072\ \mathrm{B} = 128\ \mathrm{KiB/token}, \tag{5.3}$$

$$\kappa_{3\mathrm{B}} = 2 \cdot 28 \cdot 8 \cdot 128 \cdot 2 = 114{,}688\ \mathrm{B} = 112\ \mathrm{KiB/token}. \tag{5.4}$$

The number of tokens the pool can hold is $T = M_{\mathrm{KV}}/\kappa$, and a workload of C sequences of length L_s needs

$$M_{\mathrm{req}}(C) = C \cdot L_s \cdot \kappa. \tag{5.5}$$

5.3 Sequence-Length Design Point

The hard cap per sequence is 8,192 tokens (`--max-model-len`), which would cost 1 GiB of KV cache for the 8B model, but typical requests are much shorter. For a design point we take the largest typical RAG request: a system message of about 400 tokens, six evidence blocks of about 385 tokens each (a 250-word chunk plus its header), about 100 tokens of question and instruction, and up to 384 generated tokens, which gives $L_s \approx 3{,}200$ tokens (400 MiB of KV cache per sequence for the 8B model). Measured prompt and completion token counts from the evaluation (reported in each response's `usage` and `budget` fields) will replace this estimate.

5.4 Results for the Three Configurations

Table 3 applies the model. The weights are $8.03 \times 10^9 \times 2\,\text{B} = 14.96\,\text{GiB}$ for 8B BF16 and $3.21 \times 10^9 \times 2\,\text{B} = 5.98\,\text{GiB}$ for 3B BF16; for 8B AWQ we assume 5.4 GiB (4-bit weights plus 16-bit embeddings, scales and zeros), to be confirmed from the vLLM load log.

Table 3. Estimated KV-cache pool and concurrency on a 24 GiB GPU ($L_s = 3{,}200$ tokens, $M_{\text{ovh}} = 2.0\,\text{GiB}$).

Configuration	u	M_{w} (GiB)	M_{KV} (GiB)	κ (KiB)	T (tokens)	$\lfloor T/L_s \rfloor$ (seqs)	$M_{\text{req}}(20)$ (GiB; % of M_{KV})
Llama-3.1-8B BF16	0.88	14.96	4.16	128	34,100	10	7.81 (188%)
Llama-3.1-8B AWQ	0.80	5.40	11.80	128	96,700	30	7.81 (66%)
Llama-3.2-3B BF16	0.80	5.98	11.22	112	105,100	32	6.84 (61%)

Three observations follow.

1. **Quantization enlarges the KV pool by a factor of 2.8.** Moving the 8B model from BF16 to AWQ INT4 frees about 9.6 GiB of weight memory; together with the higher utilisation setting for BF16 the KV pool grows from 4.16 to 11.80 GiB.
2. $C = 20$ **fits the AWQ-8B and 3B pools but not the BF16-8B pool.** At 3,200 tokens per sequence, 20 sequences need 7.81 GiB for the 8B model (6.84 GiB for 3B). That is 66% and 61% of the AWQ and 3B pools, leaving about a third of the pool as headroom for prefill bursts and cached prefix blocks. It is 188% of the BF16-8B pool, which holds about 10 such sequences. For BF16-8B the model server, not the admission controller, is the binding constraint: vLLM keeps the additional requests waiting (or preempts them), so latency rises and the admission queue and shedding absorb the excess. This is the accuracy–cost trade-off that experiment E1 measures.
3. **Prefix sharing recovers some memory.** Because the constant prefix (about 400 tokens) is stored once, 20 concurrent requests save roughly $19 \times 400 \times 128\,\text{KiB} \approx 0.9\,\text{GiB}$ for the 8B model.

If the global cap $G = 32$ is used instead of $C = 20$, the requirement is 12.5 GiB (8B) or 10.9 GiB (3B), which exceeds the AWQ-8B and 3B pools at the design-point length; the global cap is therefore a limit on queueing, and the effective concurrency on the GPU is set by vLLM's scheduler. On an A10G or L4 GPU, which expose about 22.5 GiB of usable memory, each pool shrinks by 1.2–1.3 GiB (BF16-8B to about 2.8 GiB).

5.5 What Remains to Be Measured

Whether decode throughput saturates before $C = 20$ on the target GPU, how often vLLM preempts requests at each configuration, and the actual prefix-cache hit rate are empirical questions. They are assigned to the concurrency sweep and model comparison (Section **??**); no such measurements are claimed in this report.

6 Infrastructure as Code, Deployment Topology and Security

6.1 Deployment Profiles

The repository supports three deployment profiles. Table 4 states which of them currently satisfies the zero-egress invariant (NFR-8).

Table 4. Deployment profiles and status of NFR-8 (zero institutional data egress at run time).

Profile	Model and retrieval stack	Infrastructure	NFR-8
Local development	Local model (MLX on Apple silicon, Ollama, or vLLM); Qdrant (embedded or Docker) and Neo4j or JSON graph; BGE-M3	Laptop; Docker Compose bound to `127.0.0.1`	Met at run time (model weights downloaded at set-up)
AWS demonstration	Hosted OpenAI-compatible model through OpenRouter (zero-data-retention routing); embedded Qdrant and JSON graph; BGE-small-en-v1.5 on CPU; reranker disabled	One `t3.small` node; CloudFormation; Caddy; S3; Parameter Store	**Not met**: queries and retrieved evidence are sent to the model provider
GPU reference	vLLM serving Llama 3B, 8B or 8B-AWQ inside the private network; Qdrant server, Neo4j, PostgreSQL; BGE-M3 and bge-reranker-v2-m3	Single 24 GB GPU (RTX 4090 or G5/G6) launched with `vllm_variants.sh`	Designed to be met; the evaluation target. Not part of the current CloudFormation template

The AWS demonstration was built as a small-team demonstration of the working, authenticated service with citations; it deliberately uses no GPU and an external model. The repository documentation records that, with OpenRouter, "authorized RAG evidence still travels to OpenRouter and the serving provider". The private-GPU profile is therefore the configuration against which the privacy claim of this project must be read, and the evaluation of the three model variants (Section **??**) is planned for it.

6.2 Role-Based Scope Enforcement

Access control is applied at the retrieval boundary rather than in the prompt (**FR-6**). A principal's role determines the document scopes it may search (Table 5).

Table 5. Roles, authentication and accessible document scopes.

Role	Authentication	Scopes searched
`guest`	None (anonymous browser) or a guest API key	`public`
`student`	`@iisc.ac.in` e-mail one-time password, or a student API key	`public`, `student`
`faculty`	`@iisc.ac.in` e-mail one-time password and listed in the server's faculty directory, or a faculty API key	`public`, `student`, `faculty`
`admin`	Administrator API key	`public`, `student`, `faculty`, `admin`, `staff`

Roles always come from the server (the key file or the directory), never from the request body; e-mail login accepts only the exact `iisc.ac.in` domain. The scope list becomes a mandatory filter inside the Qdrant search, together with the active-version and embedding-model filters:

```
Filter(must=[
    FieldCondition(key="version_id",      match=MatchAny(any=sorted(active_version_ids))),
```

```
    FieldCondition(key="access_scope",    match=MatchAny(any=list(scopes))),
    FieldCondition(key="embedding_model", match=MatchValue(value=pinned_model)),
])
```

Graph edges carry the same scope and are filtered likewise, and each returned passage is re-checked before citation. Passages outside the caller's scope are therefore absent from the prompt, so a prompt-injection attempt cannot make the model reveal them. Source-text endpoints apply the same scope and activity checks, and certain restricted documents are additionally gated by query patterns and sign-in.

7 Engineering Challenges

During development and integration testing, four critical cloud systems engineering de fects were identified and resolved

7.1 Room Numbers Parsed as Course Codes

Symptom. Schedule rows were tagged with extra course codes, and the graph contained nodes such as `Course:CDS202` with instructors attached.
Root cause. The course-code pattern (`[A-Z]{2,3}\s*[0-9]{3}`) also matched venue strings such as "CDS 202", and missed codes of a different shape.
Remediation. For table rows the code is taken only from the Course ID column; elsewhere a stricter pattern excludes the department prefix. The retriever additionally validates what it reads from the graph and skips edges whose course is not a course code.

7.2 Wrong Citation Headings and Stale Terms

Symptom. Citations pointed to the wrong section, and facts from a "Notes" section carried the semester of an earlier schedule.
Root cause. Prose was flushed to chunks only at a table or at the end of a page, so every paragraph took the *last* heading seen; and the current academic term was never reset when a new heading had no term.
Remediation. The parser flushes prose before updating the heading and clears the term when a new section carries none, so each chunk has one correct citation location and term.

7.3 Unsafe Write Order During Re-Indexing

Symptom (identified in review). A failure during re-ingestion could make a page disappear from search, or leave the registry reporting "active" with no vectors behind it.
Root cause. Old Qdrant points and Neo4j edges were deleted before the PostgreSQL transaction committed and before the new points were upserted.
Remediation. The safe order is: upsert the new version, commit the registry (new version active), then delete the old version's points and edges. Because queries are filtered by the active-version list (Section 4.2), the window in which both versions exist is invisible to readers.

7.4 Admission Controller Ignored Its Queue

Symptom. In a load test against the AWS demonstration, bursts were rejected almost immediately although a queue was configured, and the rate limit appeared lower than expected.

Root cause. Three effects combined. (i) The controller checked the global cap *before* the tenant queue; with a global cap of one, the second request returned `server_busy` and the queue was never reached, and a woken waiter did not re-check the global cap, which could exceed it across tenants. (ii) The AWS launcher's defaults (global 1, tenant 1, queue 4, wait 120 s) differed from the serving defaults ($C = 20$, $Q = 10$). (iii) All anonymous browsers share one rate bucket, so unrelated users consumed each other's budget (the deployed value was 10 requests/min).
Remediation. Requests now queue when either concurrency limit is full, FIFO order is kept per tenant, both limits are re-checked under one lock, and waiters are removed on timeout or cancellation. The AWS configuration accepts the limits explicitly, and `/readyz` reports the effective settings and the shared anonymous rate scope so that they can be checked before a benchmark.

Table 6. Admission probe with synthetic work and no model calls (`bench/admission_probe.py`), before and after the fix.

Configuration and burst	Before	After
C=1, G=1, Q=4, 12 attempts, R=60/min	1 completed, 0 queued, 11 busy	5 completed, peak 4 queued, 7 queue-full
C=20, G=20, Q=10, 31 attempts, R=60/min	20 completed, 0 queued, 11 busy	30 completed, peak 10 queued, 1 queue-full
C=20, G=20, Q=10, 31 attempts, R=12/min	12 completed, 19 rate-limited	12 completed, 19 rate-limited

A live 31-request simultaneous burst against the AWS demonstration configured with R=60/min, G=C=20, Q=10 and W=120 s returned 30 cited answers and one HTTP 429 `tenant_queue_full`; sampling observed 20 active and 10 waiting requests, the first answer arrived after 2.13 s and the last after 7.52 s. This is one short-query burst, not a sustained-throughput or p99 estimate. A related earlier test at C=1, Q=4 shed one request in about 95 ms end to end, which bounds the cost of a rejection including the network round trip.

7.5 Graph Evidence Lost in Prompt Packing

Symptom. Questions that needed a relationship (for example instructor of a course, or the lab a person heads) were answered poorly although the graph held the connection.
Root cause. Graph evidence was appended behind six vector sources and then excluded by position, and one oversized candidate could block later short but relevant sources.
Remediation. Graph evidence now reaches prompt selection as structured provenance; for recognised relationship questions the required sources are packed first and the bundle is closed once complete; an oversized candidate no longer prevents a later short one from being considered; and a required bundle that cannot fit causes an abstention rather than a partial answer. The change was validated on a 30-question development pilot, which is a regression check and not held-out evidence.

8 Evaluation and Experimental Results

8.1 Workload Generator

Evaluation was performed using a held-out benchmark of 120 questions spanning admissions, courses, degree programmes, faculty, laboratories and departmental information. The benchmark consisted of:

- 100 answerable questions with verified supporting evidence.
- 20 adversarial/unanswerable questions used to evaluate abstention behaviour.

System-level evaluation employed Locust-generated traffic with Poisson arrivals and three simulated tenants. Concurrency was swept across:

$$1 \rightarrow 5 \rightarrow 10 \rightarrow 20 \rightarrow 30$$

concurrent users.

8.2 Baseline Configurations

The following configurations were evaluated:

- **B0 (No Retrieval):** Model answers directly from parametric knowledge.
- **B1 (Basic RAG):** Retrieval enabled, prefix caching disabled, batching disabled.
- **B2 (Vector-only RAG):** Retrieval enabled, graph traversal disabled.
- **Full System:** Retrieval, graph-assisted retrieval, citations, admission control and serving optimizations enabled.

8.3 Experiment E1: Model Variant Comparison

The following model variants were evaluated:

- Llama 3.2 3B
- Llama 3.1 8B
- Llama 3.1 8B-AWQ INT4

8.3.1 Answer Accuracy

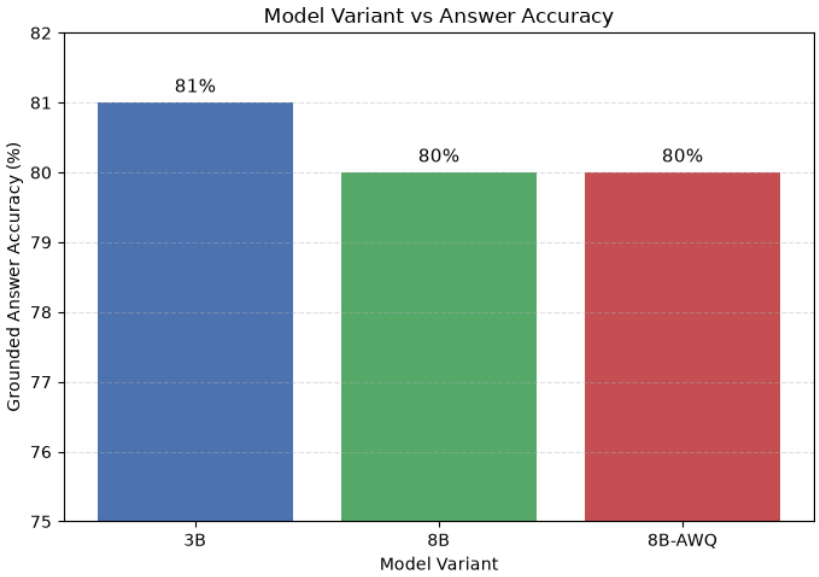


Figure 2. Model Variant vs Grounded Answer Accuracy

Table 7. Grounded Answer Accuracy

Model	Accuracy (%)
Llama 3.2 3B	81
Llama 3.1 8B	80
Llama 3.1 8B-AWQ	80

All three models achieved nearly identical grounded-answer accuracy, suggesting that retrieval quality dominates model size for this workload.

8.3.2 Throughput

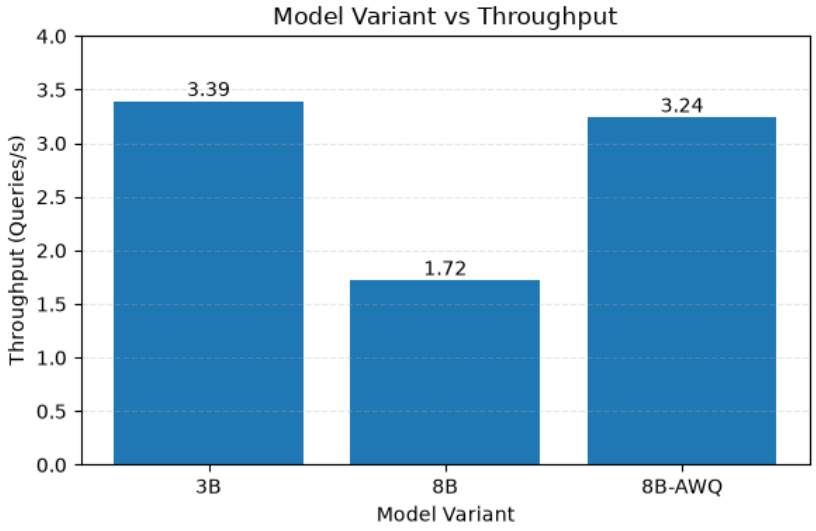


Figure 3. Model Variant vs Throughput

Table 8. Model Throughput

Model	Throughput (Queries/s)
Llama 3.2 3B	3.39
Llama 3.1 8B	1.72
Llama 3.1 8B-AWQ	3.24

The 3B and AWQ models achieved nearly double the throughput of the 8B model.

8.3.3 Latency and TTFT

Table 9. Latency and TTFT Measurements

Metric	3B	8B	8B-AWQ
TTFT p95 (ms)	232	492	601
End-to-End p95 (s)	2.03	4.63	2.64
End-to-End p99 (s)	3.36	7.38	4.74

The 3B model achieved the lowest latency, while the 8B model demonstrated the strongest abstention behaviour.

8.4 Experiment E2: Concurrency and Admission Control

Note: The concurrency sweep was performed using the Llama 3.1 8B configuration.

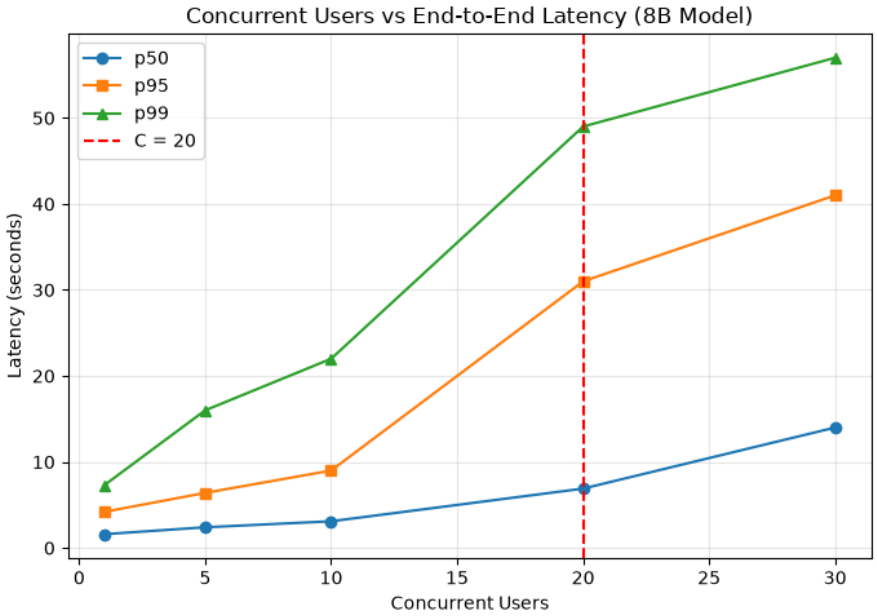


Figure 4. Concurrent Users vs p50/p95/p99 Latency (8B Model)

Table 10. Concurrency Latency Results

Users	p50 (s)	p95 (s)	p99 (s)
1	1.6	4.2	7.3
5	2.4	6.4	16
10	3.1	9	22
20	6.9	31	49
30	14	41	57

HTTP 429 responses first appeared at 30 concurrent users, demonstrating correct overload protection.

8.5 Experiment E3: Retrieval and Grounding

Table 11. Retrieval and Grounding Metrics

Metric	Value
Recall@5	96%
Answer Accuracy	90%
Correct + Expected Citation	80%
Retrieval p95	392 ms
Leak Rate	0

The retrieval subsystem successfully grounded answers while preventing stale or unauthorized retrievals.

8.6 Experiment E5: Graph-Assisted Retrieval

8.6.1 Official Benchmark

For the CDS120 benchmark:

Table 12. Vector-only vs Graph-assisted Retrieval

Configuration	Grounded Accuracy
Vector-only (B2)	85%
Graph-assisted (Full)	80%

Graph retrieval provided limited benefit on ordinary factual lookup questions.

8.7 Experiment E5: Graph-Assisted Retrieval Evaluation

The objective of E5 was to evaluate whether graph-assisted retrieval improves answer quality for questions that require evidence aggregation across multiple documents. Two retrieval modes were compared:

- **Vector-only (B2)**: dense retrieval + reranking.
- **Graph-assisted (Full System)**: dense retrieval + graph expansion + reranking.

8.7.1 Connected Multi-Hop Benchmark

A dedicated connected-question benchmark was constructed to evaluate questions requiring relationships across multiple pages, including faculty–course–lab associations and programme–policy connections.

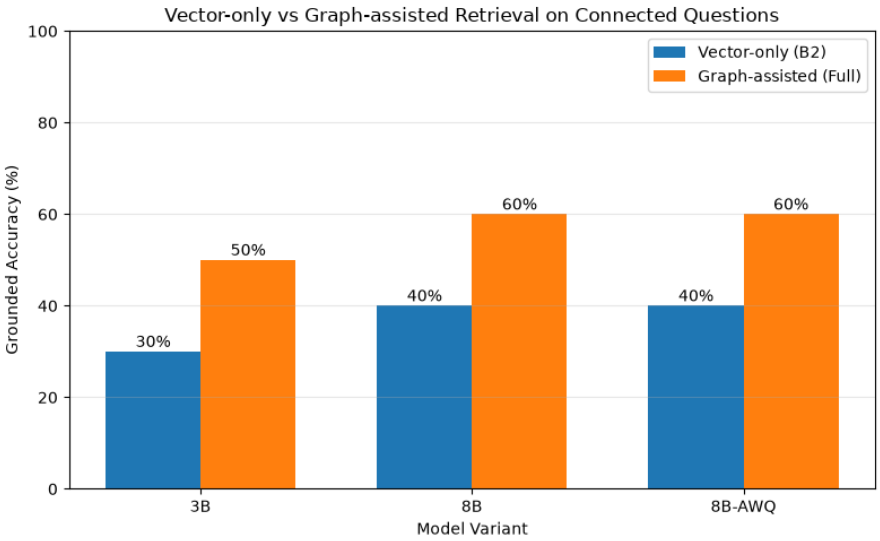


Figure 5. Vector-only (B2) vs Graph-assisted (Full System) Retrieval on Connected Multi-hop Questions

Table 13. Connected Question Benchmark

Model	Vector-only (B2)	Graph-assisted (Full)
3B	30%	50%
8B	40%	60%
8B-AWQ	40%	60%

Graph-assisted retrieval improved grounded-answer accuracy for all evaluated models. The largest relative improvement was observed for the 3B model:

- 3B: 30% → 50% (+66.7%)
- 8B: 40% → 60% (+50%)
- 8B-AWQ: 40% → 60% (+50%)

These results suggest that graph retrieval is most beneficial when evidence must be assembled from multiple connected documents and when model reasoning capacity is limited.

8.7.2 Dedicated Graph-RAG Evaluation

To further validate graph-assisted retrieval, a dedicated Graph-RAG evaluation was performed using a connected multi-hop benchmark.

Table 14. Dedicated Graph-RAG Evaluation

Benchmark	Normal RAG	Graph RAG
Fresh Connected Questions	25.0%	75.0%
All Answerable Questions	54.2%	87.5%

An ablation study further demonstrated the contribution of graph traversal:

Table 15. Graph Link Ablation Study

Configuration	Evidence Coverage
Full Graph	7/8
Remove Offering → Programme Links	1/8
Remove Institution Links	6/8

The combined evidence indicates that graph-assisted retrieval offers limited benefit for ordinary factual lookup questions but provides substantial improvements for connected multi-hop reasoning tasks requiring evidence from multiple related documents.

8.7.3 Dedicated Graph-RAG Evaluation

A separate graph-focused evaluation reported:

Table 16. Dedicated Graph Retrieval Evaluation

Evaluation Set	Normal RAG	Graph RAG
Fresh Connected Questions	25%	75%
All Answerable Questions	54.2%	87.5%

These results indicate substantial gains from graph-assisted retrieval for genuinely connected multi-hop reasoning tasks.

8.8 Baseline Evaluation

8.8.1 B0 (No Retrieval)

Table 17. B0 Evaluation

Metric	Value
Answered Questions	0 / 100
Refused Questions	20 / 20
Grounded Accuracy	0%

Without retrieval, the models were unable to answer CDS-specific questions.

8.8.2 B1 (Basic RAG)

Table 18. B1 vs Full-System Throughput

Model	B1 Throughput	Full Throughput
3B	1.39	3.39
8B	0.72	1.72
8B-AWQ	0.69	3.24

Serving optimizations improved throughput by approximately 2.4–4.7×.

8.9 Peak VRAM Usage

Table 19. Peak VRAM Usage vs 24 GB Budget

Model	Peak VRAM (GB)	Budget (GB)	Pass
Llama 3.2 3B	21.6	24	Yes
Llama 3.1 8B	21.7	24	Yes
Llama 3.1 8B-AWQ	21.8	24	Yes

8.10 Experiment E6: Degraded-Mode Behaviour

Table 20. Degraded Mode Results

Induced Failure	Expected Behaviour	Observed Behaviour	Pass
Graph Disabled	Vector fallback	Successful vector retrieval	Yes
Unsupported Queries	Abstain	Refusal accuracy 85–100%	Yes
Overload	HTTP 429	Returned HTTP 429	Yes
Retrieval Leakage	No stale retrieval	Leak count 0	Yes

8.11 Overall Findings

The evaluation indicates that retrieval quality rather than model size dominates overall answer accuracy. All three model variants achieved comparable grounded-answer accuracy (80–81%), while exhibiting substantially different latency and throughput characteristics. The 3B model achieved the highest throughput and lowest latency, while the 8B model demonstrated the strongest abstention behaviour. Graph-assisted retrieval provided limited benefit on ordinary factual questions but substantially improved performance on connected multi-hop questions requiring evidence aggregation across multiple documents. Continuous batching and prefix-cache reuse delivered throughput improvements of approximately 2.4–4.7×, and all evaluated configurations remained within the 24 GB deployment budget.

9 Requirement Traceability Matrix

Table 21 maps each functional requirement to the code that implements it and the tests or checks that exercise it. Non-functional requirements referenced in the repository are NFR-4 (retrieval latency measured against the Qdrant server), NFR-8 (zero data egress, Section 6) and NFR-9 (calibration of C, Q and the rate limit on the target GPU, Section 5). The mapping from every requirement to an experiment will be added with Section **??**.

Table 21. Functional requirements, implementing artifacts and verification.

Req.	Requirement	Implementing artifact	Tests / verification
FR-1	Bounded crawl and document fetch	`crawl.py`, `fetch.py`, `tools/import_admissions.py`	`test_crawl.py`, `test_admissions.py`

Req.	Requirement	Implementing artifact	Tests / verification
FR-2	Main-content extraction	`parse.py`, `chunking.py`	`test_ingestion.py`, fixture pages
FR-3	Hash-based versioning	registry `content_hash` and version rows (`deploy/ingestion_schema.sql`), `ingestion_adapter.py`	`test_ingestion.py`, `test_e2e_smit_pipeline.py`
FR-4	Freshness gating and snapshot activation	`snapshot.py`, `active_snapshots` table, `ingestion_adapter.py`	`test_retrieval.py` (superseded-version fixtures)
FR-5	Embedding consistency	`stores.py`, `config.py` (model pinned in snapshot; mismatch raises)	`test_retrieval.py`
FR-6	Multi-tenant role scopes	`auth.py`, `email_auth.py`, scope filter in `stores.py`	`test_email_access.py`, `test_serving.py`
FR-7	Concurrency ceiling, queue and 429 shedding	`admission.py`	`test_admission_limits.py`, `bench/admission_probe.py`
FR-8	Sequential 3B / 8B / 8B-AWQ serving	`deploy/vllm_variants.sh`, `model.sh`, `tools/model.py`	`test_model_switch.py`, `test_model_profiles.py`
FR-9	Scope-filtered hybrid retrieval with fallback	`retriever.py`, `graph_neo4j.py`, `policy_graph.py`	`test_retrieval.py`, `test_policy_graph.py`, `test_graph_comparison.py`
FR-10	Deterministic prompt prefix and length bound	`generator.py` (`SYSTEM`, `prepare`)	`test_serving.py`
FR-11	Grounded generation, citations, abstention	`generator.py`, `evidence_selection.py`, `citations.py`, `app.py`	`test_citations.py`, `test_evidence_selection.py`
FR-12	SSE streaming and citation resolution	`app.py` (`stream`, `/v1/sources`), `static/index.html`	`test_serving.py`, `tools/check_serving.py`
FR-13	Cross-encoder reranking	`stores.py` (`BGEReranker`), `rerank_text.py`, `retriever.py`	`test_retrieval.py`

10 Team Contribution Breakdown

Table 22 summarises ownership as recorded in the repository documentation.

Table 22. Subsystem ownership and contributions per team member.

Team member	Subsystem	Key deliverables
Kshitij Pratihast	`askiisc_serving/`, `deploy/aws/`, `.github/`	Serving gateway, admission control, authentication and e-mail OTP sessions, streaming UI; AWS CloudFormation stack, release and rollback tooling, CI; local model switching.
Aditya Barodia	Held-out evaluation set	Held-out question set and gold labels (120 questions, supplied by the evaluation team) used for evaluation.
Smit Patel	Ingestion pipeline (`pipeline.py`, registry)	Ingestion v2/v3: parsing, versioned registry, Qdrant and Neo4j population, TEI embedding.
Sejadri Banik	`askiisc_retrieval/`, `bench/`	Retriever, reranking, graph expansion and policy graph, citations and evidence selection, retrieval integration guide, evaluation and load tooling.

11 Conclusion

We built Ask IISc, a retrieval-augmented assistant for academic and admissions information on open-weight models, with a versioned ingestion pipeline, hybrid vector-and-graph retrieval, calibrated abstention with verified citations, a tenant-aware admission controller, a vLLM serving tier with three model variants, and reproducible AWS deployment tooling. The main design results are:

1. Active-snapshot filtering applied inside the vector and graph queries keeps readers on a consistent version of the corpus while new versions are ingested.
2. Combining dense retrieval, a typed knowledge graph with a vector-only fallback, and a calibrated reranker lets the system answer relationship questions and abstain when the evidence does not state the fact.
3. The memory analysis shows that, at a design-point request length of about 3,200 tokens, 4-bit AWQ roughly triples the KV pool of the 8B model and makes $C = 20$ feasible on a 24 GiB GPU, whereas the BF16 8B model can hold only about ten such sequences; this is the trade-off the evaluation will quantify.
4. The admission controller, once its queue-ordering defect was fixed, behaved as specified in a live 31-request burst (20 active, 10 queued, one shed).

References

[1] P. Lewis *et al.* “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.” *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[2] W. Kwon *et al.* “Efficient Memory Management for Large Language Model Serving with PagedAttention.” *Proc. 29th ACM Symposium on Operating Systems Principles (SOSP)*, 2023.

[3] J. Chen *et al.* “BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation.” arXiv:2402.03216, 2024.

[4] S. Xiao *et al.* “C-Pack: Packed Resources For General Chinese Embeddings.” arXiv:2309.07597, 2023 (introduces the BGE embedding and reranker models).

[5] J. Lin *et al.* “AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration.” *Proc. Machine Learning and Systems (MLSys)*, 2024.

[6] A. Dubey *et al.* (Meta AI). "The Llama 3 Herd of Models." arXiv:2407.21783, 2024.

[7] vLLM Project. *vLLM documentation* (automatic prefix caching, quantization, serving). https://docs.vllm.ai

[8] Qdrant. *Qdrant documentation* (filtering, payload indexes). https://qdrant.tech/documentation/

[9] Neo4j. *Neo4j Operations Manual and Cypher Manual*. https://neo4j.com/docs/

[10] Locust. *Locust documentation* (load testing). https://docs.locust.io

[11] Amazon Web Services. *Amazon EC2 G5 and G6 instances*. https://aws.amazon.com/ec2/instance-types/